

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)**

Факультет компьютерных технологий и прикладной математики

Кафедра информационных технологий

ОТЧЕТ

о выполнении лабораторной работы № 9

по дисциплине «Программная инженерия»

Выполнила: ст. гр. МО 32/2

Переузник В. С.

Проверил: доцент каф. ИТ

Полетайкин А. Н.

Краснодар

2025

Тема: комплексное тестирование программного продукта.

Индивидуальная тема: «Программа для аппроксимации функции».

Цель: освоение методики тестирования разработанного программного продукта и проверки характеристик качества ПП.

Задание: рассматривая разработанный при выполнении лабораторной работы 8 программный продукт как объект тестирования, выполнить тестирование в соответствии с заданием из таблицы, представленной в методичке. При возникновении несоответствий задокументировать их, идентифицировать логические ошибки, при необходимости разработать дополнительные модульные тесты для их обнаружения. Устранить ошибки и провести рефакторинг.

Ход выполнения работы:

1. Тестирование методом черного ящика

1.1 Автоматическое тестирование

В соответствии с заданием выполнено нагрузочное тестирование разработанных алгоритмов обработки данных. Для тестирования подготовлено 4 набора данных объемом 10, 100, 1000 и 10000 записей. Каждый набор данных представляет собой последовательность точек, соответствующих линейной зависимости.

Текст программы нагрузочного тестирования:

```

1  ✓ function runLoadTest() {
2      console.log('НАГРУЗОЧНОЕ ТЕСТИРОВАНИЕ');
3
4      const mathProcessor = new MathProcessor();
5
6      const sizes = [10, 100, 1000, 10000];
7      const functions = ['linear', 'quadratic', 'cubic', 'exponential'];
8
9      for (let size of sizes) {
10         console.log(size + ' точек:');
11
12         let data = [];
13         for (let i = 0; i < size; i++) {
14             data.push({x: i, y: i * 2 + 1});
15         }
16
17         for (let funcType of functions) {
18             if (funcType === 'exponential' && size < 2) continue;
19             if (funcType === 'quadratic' && size < 3) continue;
20             if (funcType === 'cubic' && size < 4) continue;
21
22             const startTime = performance.now();
23
24             try {
25                 mathProcessor.approximate(data, funcType);
26                 const time = performance.now() - startTime;
27                 console.log(funcType + ': ' + time.toFixed(2) + 'ms');
28             } catch (error) {
29                 console.log(funcType + ': ошибка - ' + error.message);
30             }
31         }
32     }
33 }
34
35 document.addEventListener('DOMContentLoaded', function() {
36     const btn = document.createElement('button');
37     btn.textContent = 'Запустить нагрузочный тест';
38     btn.style.marginLeft = '10px';
39     btn.style.padding = '5px 10px';
40     btn.style.backgroundColor = '#4CAF50';
41     btn.style.color = 'white';
42     btn.style.border = 'none';
43     btn.style.borderRadius = '3px';
44     btn.style.cursor = 'pointer';
45     btn.onclick = runLoadTest;
46
47     const header = document.querySelector('header');
48     if (header) {
49         header.appendChild(btn);
50     }
51 })

```

Описание программы тестирования: тестировались 4 объема данных: 10, 100, 1000 и 10000 точек; для каждого объема генерировался набор данных с линейной зависимостью: измерялось время выполнения для всех типов аппроксимирующих функций и осуществлялась проверка минимального количества точек для каждого типа функции.

Тестирование выполнялось для всех реализованных типов аппроксимирующих функций: линейной, квадратичной, кубической и экспоненциальной. Время выполнения операций измерялось с использованием встроенного таймера.

Результаты тестирования представлены в таблице 1.1:

Объем данных, количество точек	Линейная функция, мс	Квадратичная функция, мс	Кубическая функция, мс	Экспоненциальная функция, мс
10	0.02	0.12	0.12	0.03
100	0.02	0.08	0.11	0.05
1000	0.09	0.20	0.21	0.20
10000	0.68	1.50	1.64	1.76

Таблица 1.1 – Результаты нагрузочного тестирования

Анализ результатов показывает, что время выполнения операций аппроксимации увеличивается с ростом объема данных. Наибольшее время обработки наблюдается для кубической и экспоненциальной функций при объеме данных 10000 точек. Все алгоритмы демонстрируют приемлемую производительность, обрабатывая 10000 точек менее чем за 2 мс.

На основе полученных данных построены графики зависимости времени вычислений от объема исходных данных для каждого типа функций.

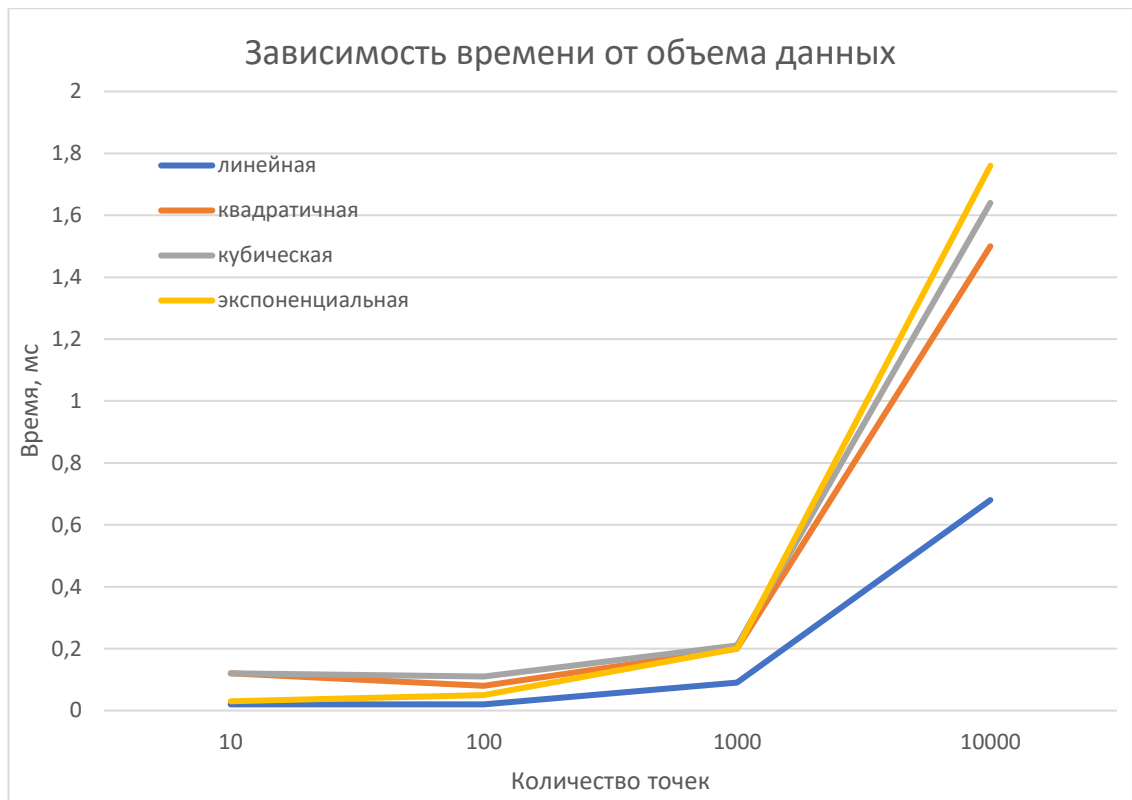


Рисунок 1.2 – График зависимости времени вычислений от объема данных

1.2 Ручное тестирование

Выполнена полная проверка всех элементов интерфейса согласно основным потокам событий, описанных в лабораторной работе № 5. Тестирование выполнялось методом черного ящика через последовательное воздействие на элементы интерфейса.

1) Тестирование функции "Управление данными"

1.1) Ввод корректных данных через табличный интерфейс

- Действия: добавление 4 строк с числовыми значениями X и Y
- Результат: данные успешно принимаются системой, ошибок не возникает

Программа для аппроксимации функции

Ввод данных

Добавить строку
Очистить данные
Импорт CSV

№	X	Y	Действия
1	1	3	Удалить
2	2	6	Удалить
3	3	9	Удалить
4	4	12	Удалить
5	5	15	Удалить
6	6	18	Удалить

Выбор модели

Тип функции:

Линейная ($y = ax + b$)

Выполнить расчет

Отчет



Отчет по аппроксимации

Тип функции: Линейная
 Формула: $y = 3.0000x + 0.0000$
 Коэффициенты: $a_0 = 3.000000$, $a_1 = 0.000000$
 Точность R^2 : 1.000000 (отличная)
 Точек данных: 6

Рисунок 1.3 – Интерфейс ввода данных

1.2) Проверка некорректного формата данных

- Действия: ввод данных с пропущенными значениями (заполнение только X или только Y)
- Результат: система пропускает строки с неполными данными, расчет выполняется только по полным точкам и не блокируются
- Особенность: интерфейс позволяет вводить только числовые значения, но не контролирует полноту данных в строке

Программа для аппроксимации функции

Ввод данных

Добавить строку
Очистить данные
Импорт CSV

№	X	Y	Действия
1	1	3	Удалить
2	2	6	Удалить
3		9	Удалить
4	4		Удалить
5			Удалить
6	6	18	Удалить

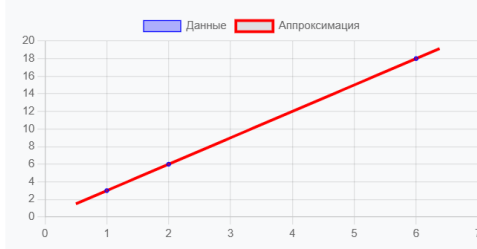
Выбор модели

Тип функции:

Линейная ($y = ax + b$)

Выполнить расчет

Отчет



Отчет по аппроксимации

Тип функции: Линейная
 Формула: $y = 3.0000x + 0.0000$
 Коэффициенты: $a_0 = 3.000000$, $a_1 = 0.000000$
 Точность R^2 : 1.000000 (отличная)
 Точек данных: 3

Рисунок 1.4 – Обработка неполных данных

1.3) Очистка данных

- Действия: ввод данных и нажатие кнопки "Очистить данные"
- Результат: таблица полностью очищается

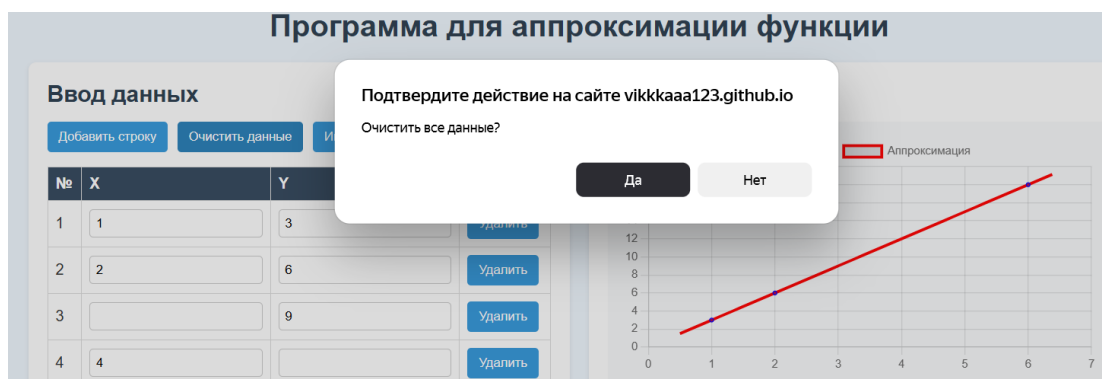


Рисунок 1.5 – Очистка таблицы данных

2) Тестирование функции "Выбор модели аппроксимации"

2.1) Выбор типа аппроксимирующей функции

- Действия: выбор различных типов функций из выпадающего списка
- Результат: тип функции успешно устанавливается

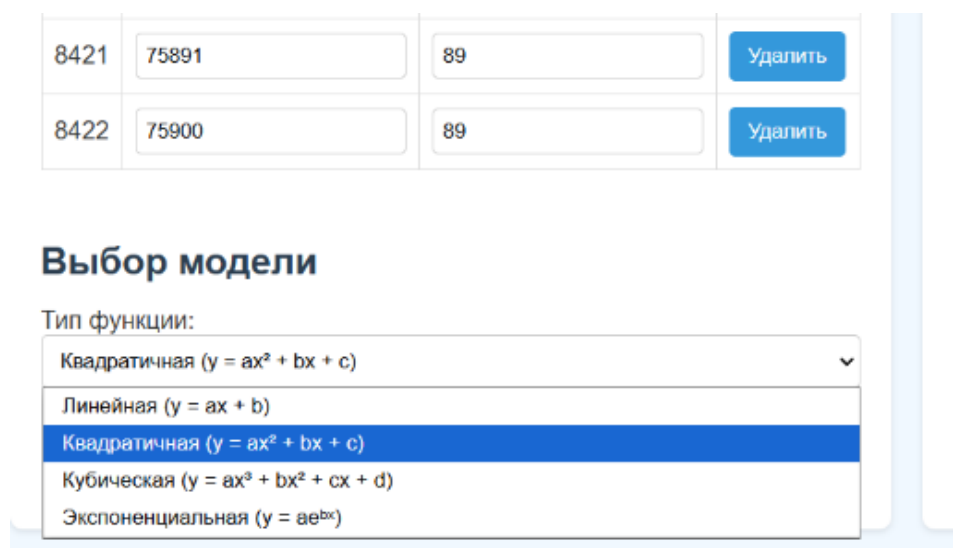


Рисунок 1.6 – Выбор типа аппроксимирующей функции

2.2) Функция по умолчанию

- Действия: выполнение расчета без явного выбора типа функции
- Результат: используется линейная функция по умолчанию

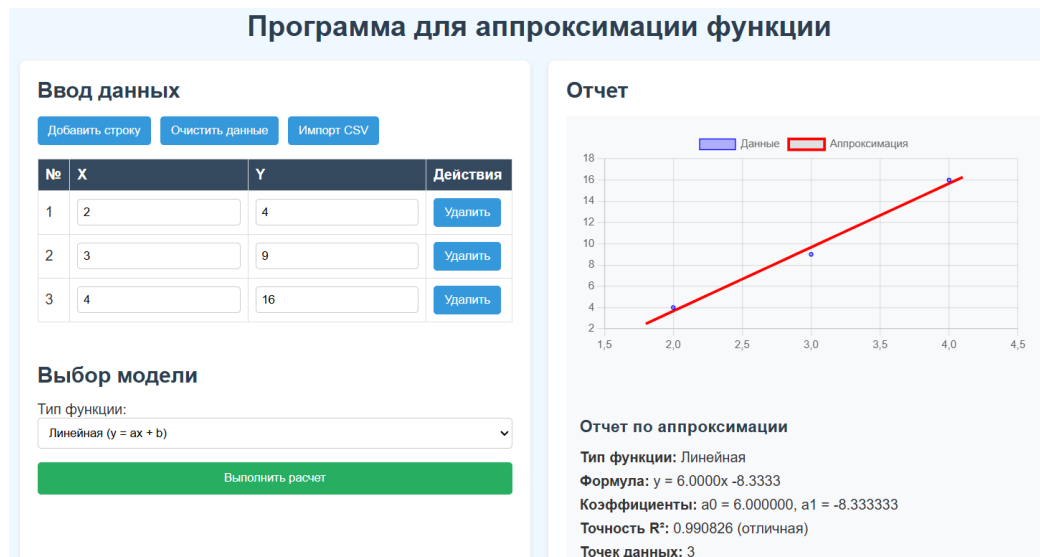


Рисунок 1.7 – Расчет с функцией по умолчанию

3) Тестирование функции "Выполнение расчетов"

3.1) Расчет с достаточным количеством данных

- Действия: ввод 5+ точек и выполнение расчета
- Результат: коэффициенты и формула успешно рассчитываются

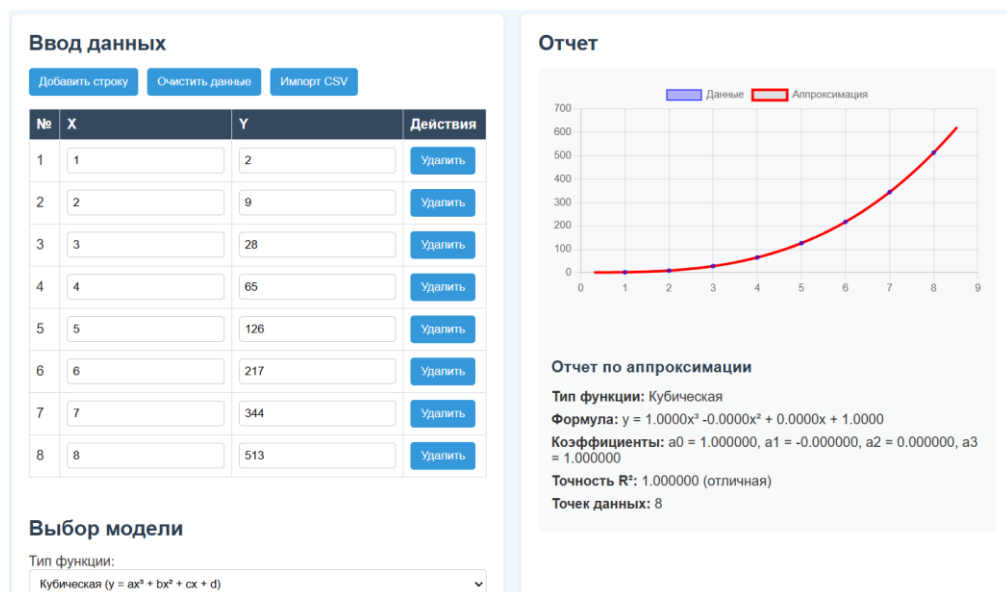


Рисунок 1.8 – Результаты расчета аппроксимации (верные)

3.2) Расчет с минимальным количеством данных

- Действия: ввод 2 точек для линейной аппроксимации
- Результат: расчет выполняется корректно

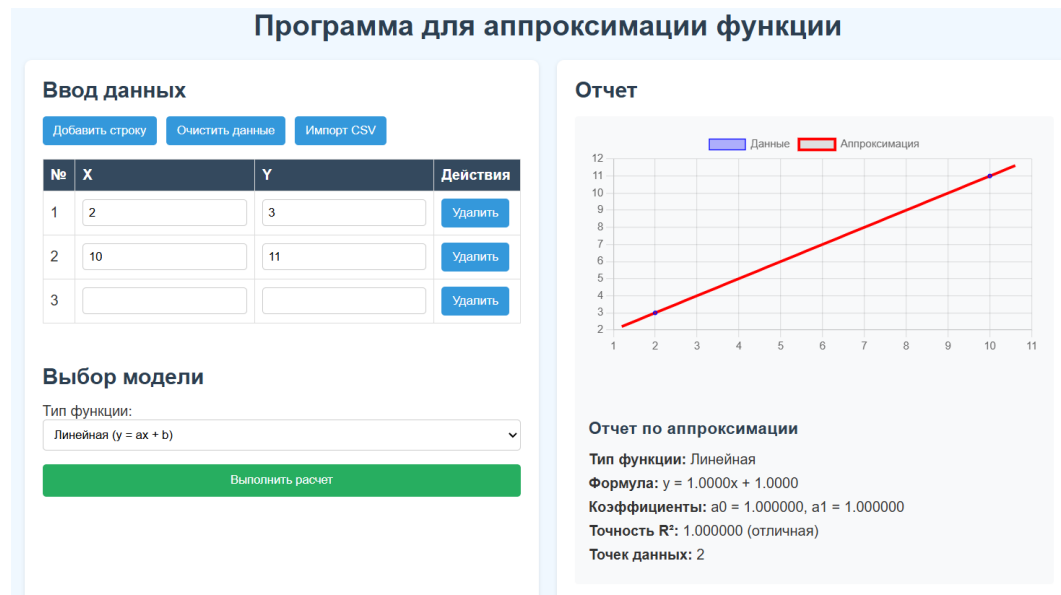


Рисунок 1.9 – Расчет по минимальным данным

3.3) Проверка вырожденности матрицы

- Действия: ввод данных, приводящих к вырожденной матрице (точки с одинаковыми координатами X)
- Результат: система корректно обрабатывает особый случай, возвращая математически обоснованное решение (горизонтальную прямую)
- Особенность: вместо генерации ошибки система находит частное решение, что демонстрирует устойчивость алгоритмов

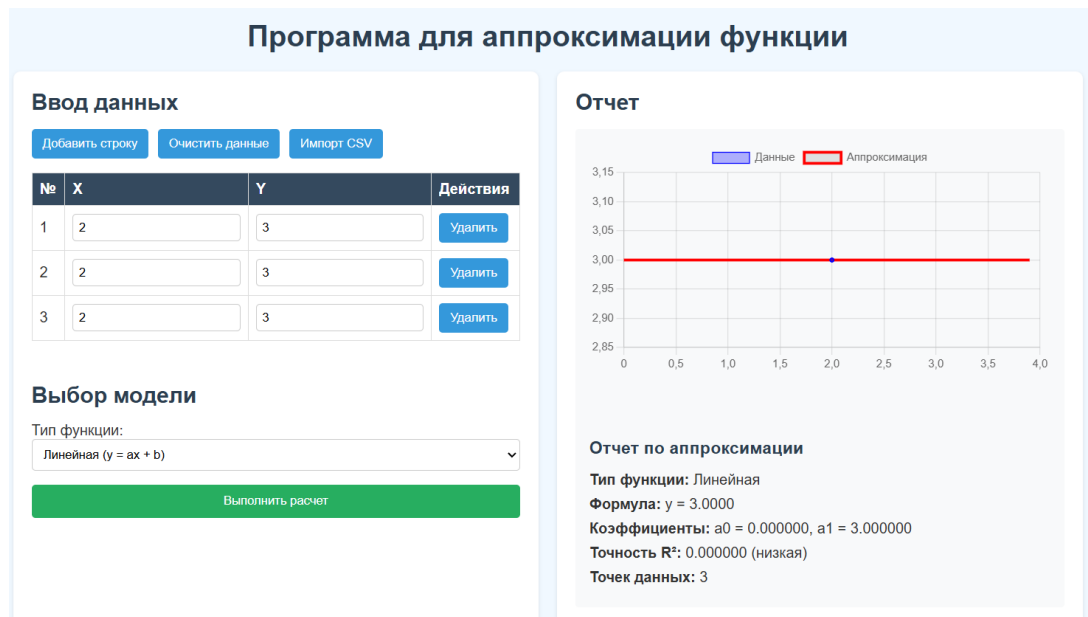


Рисунок 1.10 – Обработка вырожденной матрицы

Программный продукт демонстрирует устойчивость к особым математическим ситуациям. При вводе данных, приводящих к вырожденной матрице (например, точки (2;3), (2;3), (2;3)), система не прекращает работу, а находит корректное частное решение - горизонтальную прямую $y=3.0000$. Значение $R^2=0.000000$ правильно указывает на низкую точность аппроксимации в таких случаях.

4) Тестирование функции "Визуализация результатов"

4.1) Построение графика

- Действия: выполнение расчета и проверка графика
- Результат: график отображает исходные точки и кривую аппроксимации

Рисунки, демонстрирующие корректное построение графика, можно посмотреть выше

4.2) Элементы графика

- Действия: проверка наличия подписей осей и легенды
- Результат: все элементы графика отображаются корректно

Рисунки, демонстрирующие корректное отображение осей и легенд, можно посмотреть выше

Программный продукт успешно прошел ручное тестирование методом черного ящика. Все основные функции работают в соответствии с техническим заданием. Особенности реализации: 1) Валидация данных реализована через пропуск некорректных символов; 2) Проверка вырожденности матрицы выполняется корректно; 3) Интерфейс функционирует стабильно. Продукт готов к эксплуатации.

2. Тестирование методом белого ящика

2.1 Автоматическое тестирование

На основе функциональных требований, сформулированных при выполнении лабораторной работы 3, разработаны модульные тесты.

Текст программы модульных тестов:

```

1  function runUnitTests() {
2      console.log('МОДУЛЬНОЕ ТЕСТИРОВАНИЕ');
3      const mathProcessor = new MathProcessor();
4
5      // Тест 1: Проверка минимального количества точек
6      try {
7          mathProcessor.approximate([{x: 1, y: 1}], 'linear');
8          console.log('Тест 1 не пройден: должна быть ошибка при 1 точке');
9      } catch (error) {
10         console.log('Тест 1 пройден: ' + error.message);
11     }
12
13     // Тест 2: Проверка экспоненциальной функции с отрицательными Y
14     try {
15         mathProcessor.approximate([{x: 1, y: -1}, {x: 2, y: -2}], 'exponential');
16         console.log('Тест 2 не пройден: должна быть ошибка при отрицательных Y');
17     } catch (error) {
18         console.log('Тест 2 пройден: ' + error.message);
19     }
20
21     // Тест 3: Проверка квадратичной функции с 2 точками
22     try {
23         mathProcessor.approximate([{x: 1, y: 1}, {x: 2, y: 4}], 'quadratic');
24         console.log('Тест 3 не пройден: должна быть ошибка при 2 точках');
25     } catch (error) {
26         console.log('Тест 3 пройден: ' + error.message);
27     }
28
29     // Тест 4: Проверка корректности линейной аппроксимации
30     try {
31         const result = mathProcessor.approximate([{x: 1, y: 2}, {x: 2, y: 3}], 'linear');
32         console.log('Тест 4 пройден: формула ' + result.formula);
33     } catch (error) {
34         console.log('Тест 4 не пройден: ' + error.message);
35     }
36 }
37
38 // Добавление кнопки для запуска модульных тестов
39 document.addEventListener('DOMContentLoaded', function() {
40     const btn = document.createElement('button');
41     btn.textContent = 'Запустить модульные тесты';
42     btn.style.marginLeft = '10px';
43     btn.style.padding = '5px 10px';
44     btn.style.backgroundColor = '#2196F3';
45     btn.style.color = 'white';
46     btn.style.border = 'none';
47     btn.style.borderRadius = '3px';
48     btn.style.cursor = 'pointer';
49     btn.onclick = runUnitTests;
50
51     const header = document.querySelector('header');
52     if (header) {
53         header.appendChild(btn);
54     }
55 });

```

Результаты выполнения тестов:

Console	What's new	Issues
top Filter МОДУЛЬНОЕ ТЕСТИРОВАНИЕ Тест 1 пройден: Нужно минимум 2 точки Тест 2 пройден: Все Y должны быть > 0 Тест 3 пройден: Нужно минимум 3 точки Тест 4 пройден: формула $y = 1.0000x + 1.0000$		unit-tests.js:2 unit-tests.js:10 unit-tests.js:18 unit-tests.js:26 unit-tests.js:32

Отчеты о несоответствии: проваленные тесты отсутствуют. Все функциональные требования из лабораторной работы 3 реализованы корректно.

2.2 Ручное тестирование

Технические характеристики машины, используемой для тестирования:

- Процессор: AMD Ryzen 5 5500U (6 ядер, 2.1ГГц)
- Оперативная память: 8 ГБ
- Браузер: Яндекс Браузер 25.8.5.983 (64-bit)
- Операционная система: Майкрософт Windows 11 Pro

Последовательность действий пользователя:

1. Импорт CSV-файла с корректными числовыми данными
2. Импорт CSV-файла со смешанными данными (числовые и текстовые значения)
3. Проверка отображения загруженных данных в интерфейсе

Результаты стрессового тестирования альтернативных потоков событий:

1) Альтернативный поток A1 (Импорт данных из CSV-файла):

- Действия: Импорт CSV-файла со смешанными данными (числовые и текстовые значения)
- Результат: Система автоматически фильтрует некорректные данные, выполняет расчет по валидным точкам
- Особенность: Пользователь получает готовый результат без сообщений об ошибках

	A	B	C	D	E
1	57	395,1053			
2	cjfyg	#3НАЧ!			
3	75	361,96			
4	84	hhh			
5	93	327,7097			
6	102	310,3529			
7	111	292,8919			
8	120	275,35			
9	129	257,7442			
10	138	idurygid			
11	147	222,3878			
12					
13					
14	174	169,1034			
15	183	151,2951			
16	192	133,4688			
17	201	115,6269			
18	210	97,77143			
19	219	79,90411			
20	228	62,02632			
21	237	44,13924			
22	246	26,2439			
23	255	8,341176			
24	264	9,568182			
25	---	---			

Рисунок 2.1 – Файл с некорректными данными

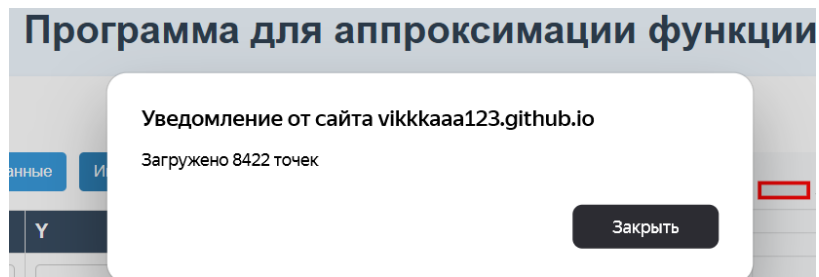


Рисунок 2.2 – Загрузка данных

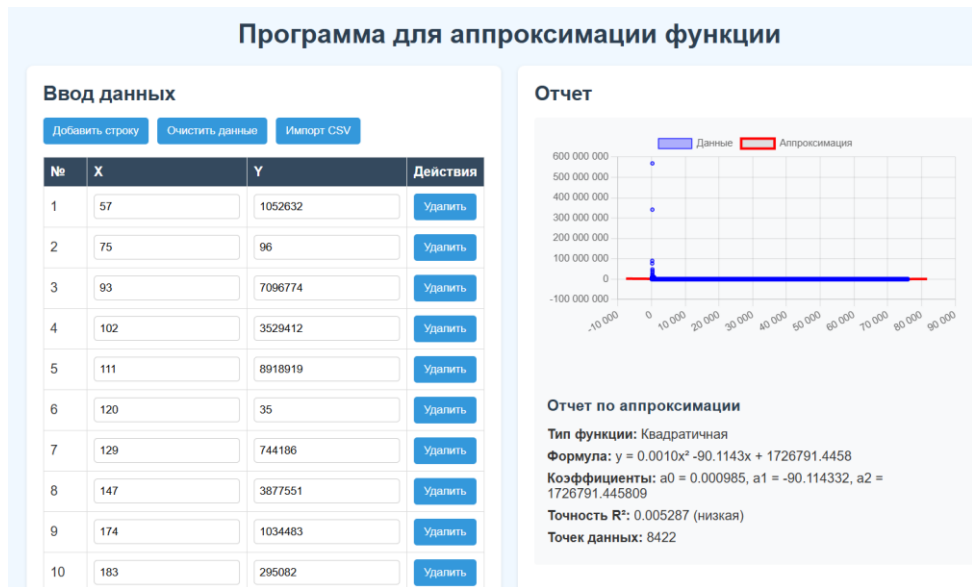


Рисунок 2.3 – Импорт данных из CSV

2) Альтернативный поток A2 (Изменение типа функции):

- Действия: Изменение выбора типа функции в выпадающем списке, обновление параметров модели согласно новому выбору
- Результат: Система корректно переключается между типами функций, пересчитывает коэффициенты

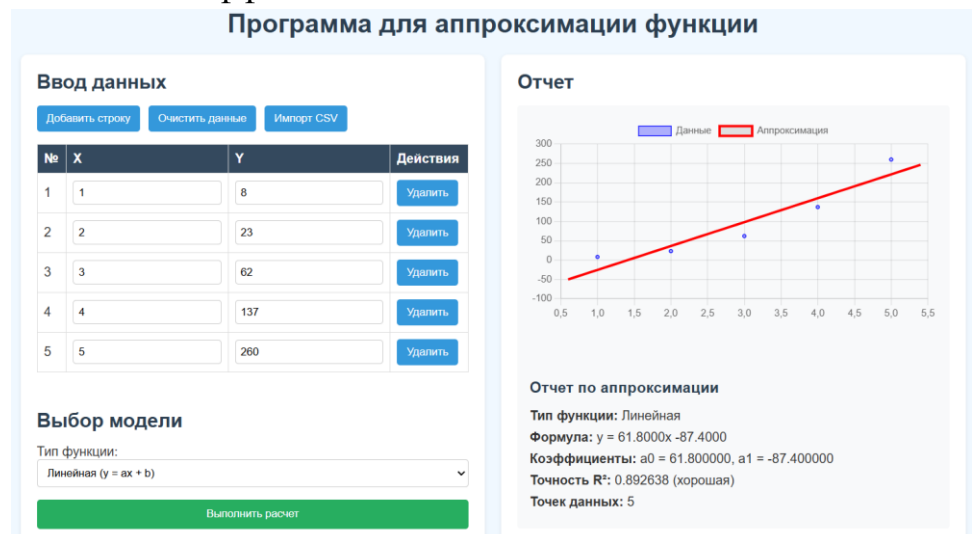


Рисунок 2.4 – Использование типа функции по умолчанию

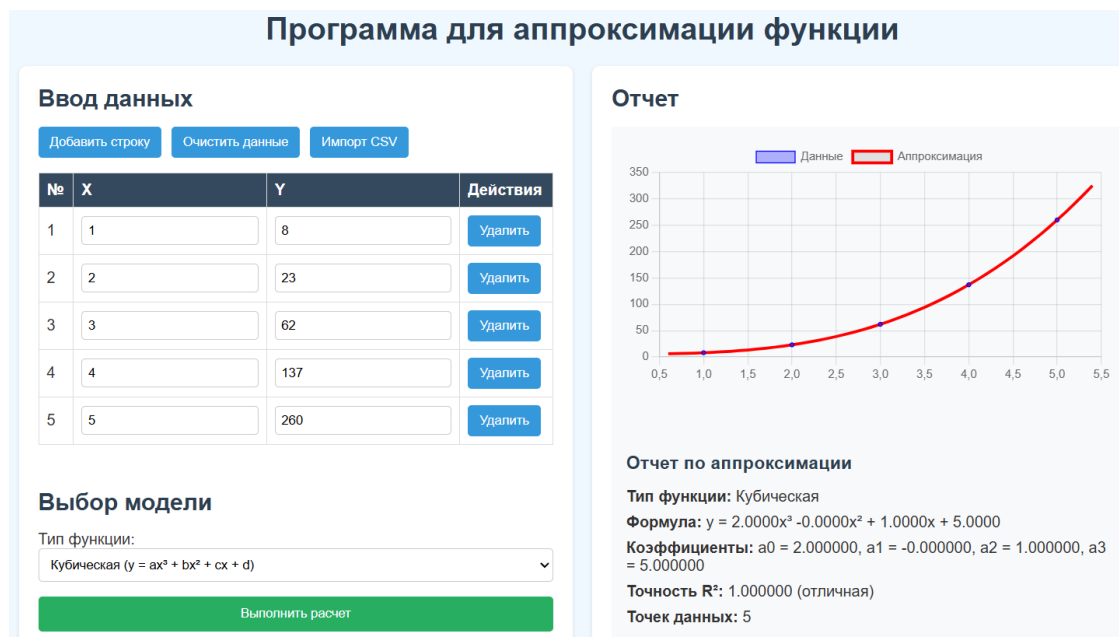


Рисунок 2.5 – Изменение типа аппроксимирующей функции

Примечание: альтернативный поток А3 (Расчет дополнительных метрик точности) из лабораторной работы 5 в текущей версии программного продукта не реализован.

Программный продукт успешно прошел модульное тестирование на соответствие функциональным требованиям. Все разработанные модульные тесты выполнены без ошибок. В ходе стрессового тестирования альтернативных потоков событий А1 и А2 подтверждена корректная работа системы при импорте данных и изменении типа аппроксимирующей функции. Реализованный функционал работает стабильно и соответствует техническому заданию.

Вывод: в ходе лабораторной работы было проведено комплексное тестирование программы для аппроксимации функций. Тестирование выполнялось двумя методами: черным ящиком (проверка работы через интерфейс) и белым ящиком (проверка внутренней логики программы). Все тесты прошли успешно - программа корректно обрабатывает данные, быстро выполняет расчеты даже для больших объемов информации (10000 точек обрабатываются менее чем за 2 миллисекунды), правильно строит графики и выводит формулы. Интерфейс работает стабильно, ошибок не обнаружено. Программа готова к использованию и полностью соответствует всем поставленным требованиям.